

Fantavega re-audit: timer di risposta, sicurezza e performance

Data: 9 agosto 2026

Revisione pubblica verificata: 3443141ff85b9a2756ff52edab4eb885864fb35b (main)

Commit applicativo precedente: 31829a7dd12f69ff57822f1c67c41447e3ac21f8

Repository: <https://github.com/nuno80/fantavega>

Verdetto esecutivo

Il disegno corretto è quello descritto: presenza separata dalla visualizzazione, claim atomico per `user + league + auction`, e timer avviato solo quando la card di risposta è realmente montata per il proprietario.

Ma questa correzione non risulta ancora presente nel `main` pubblico verificato. Il commit più recente aggiunge documentazione; il codice in `main` contiene ancora `activateTimersForUser` dentro entrambe le API di polling. Quindi il bug cruciale è ancora riproducibile sul codice pubblico, indipendentemente dal fatto che il fix esista in un branch locale o non ancora pubblicato.

Problema 1: timer avviato prima della visualizzazione

Stato osservato

`src/app/api/user/auction-states/route.ts` esegue:

```
updateHeartbeat(user.id).catch(...);
activateTimersForUser(user.id).catch(...);
```

La funzione non riceve lega o asta e attiva tutti i timer `pending` dell'utente. Il secondo endpoint, `src/app/api/leagues/[league-id]/auction-state/route.ts`, nella revisione applicativa 31829a7 passa anch'esso da `heartbeat` a `activateTimersForUser(user.id, heartbeatAt)`.

Questo è **High severity**: aprire o aggiornare la lega A può attivare il timer della lega B; reconnect, polling e rete instabile possono consumare l'ora senza una conferma di visualizzazione.

Controllo del percorso corretto

Il repository contiene già `response-timer-view.service.ts` e la route:

```
POST /api/leagues/:leagueId/players/:playerId/response-timer/viewed
```

Il claim verifica autenticazione, accesso alla lega, asta attiva, utente, lega e stato `pending`, poi usa un `UPDATE` condizionale. Questo è il confine giusto e impedisce doppie attivazioni concorrenti.

Gap residuo lato client

Nel codice pubblico verificato, `ResponseActionModal.tsx` chiama `viewed` quando il modal si apre. È meglio di un'attivazione durante il polling, ma non è la garanzia più forte: la card `ResponseNeededSlot` è

già montata prima che l'utente apra il modal, e il requisito di gioco è "vista della card", non "click su Abbandona/Rilancia".

La patch A sposta la conferma nel componente che rende la card per l'utente corrente. La chiamata deve essere idempotente e non deve partire per card read-only di altri manager.

Matrice edge case

Scenario	Risultato corretto dopo patch A
Apertura lega A con timer pendente in lega B	Timer B resta <code>pending</code>
Refresh o polling ripetuto	Nessun nuovo deadline
Reconnect Socket.IO	Nessuna attivazione
Card visibile in due tab	Un solo claim, una sola transizione DB
Altro utente osserva la tua card	Nessun claim per il tuo timer
POST viewed su lega diversa	403, nessuna scrittura
Asta non più active	Claim rifiutato
Chiusura pagina prima del mount	Timer resta <code>pending</code>
Timeout HTTP della conferma	Timer resta <code>pending</code> , retry al remount

Problema 2: cooldown e preferenze

La tabella `user_player_preferences` condivide preferiti, filtri e cooldown sulla chiave (`user_id`, `player_id`, `league_id`). `INSERT OR REPLACE` è pericoloso perché cancella la riga e la reinserisce: i campi personali non inclusi nella `INSERT` tornano ai default.

La correzione deve essere verificata in **tutti** i writer, non solo nel processor principale: expiry timer, abandon volontario e il vecchio percorso `auction-states.service.ts`. Il pattern corretto è `INSERT ... ON CONFLICT DO UPDATE`, aggiornando soltanto `preference_type`, `expires_at` e `updated_at`.

La migrazione storica `add_user_player_preferences.sql` non contiene tutti i campi usati dal codice moderno. `CREATE TABLE IF NOT EXISTS` non aggiunge colonne a una tabella esistente: serve una migrazione esplicita e tracciata, non una correzione opportunistica durante ogni startup.

Problema 3: autorizzazioni

La protezione per `user/auction-states`, la preferenza personale `toggle-icon` e il processor scoped per lega risultano concettualmente corretti nel commit applicativo 31829a7.

Va però verificato il branch finale che dichiara di aver modificato: le server actions `updateTeamNameAction`, `updateLeagueStatusAction` e `updateActiveRolesAction` devono chiamare `checkIsAdmin` dentro la server action, prima di qualunque query o mutazione. Il middleware e il fatto che il pulsante sia nascosto non bastano.

Performance e regressioni residue

Il batching dei cooldown in `/api/players` elimina il problema N+1. Restano tre rischi:

- il rate limiter in-memory non è condiviso tra istanze;
- la lease dello scheduler dura 45 secondi e non ha rinnovo;
- il bulk lookup senza `leagueId` può collassare cooldown dello stesso giocatore provenienti da leghe diverse nella stessa `Map<number, ...>`.

Stato CI

Il workflow database del commit `31829a7` risultava verde. Il workflow applicativo era ancora in esecuzione al momento del controllo e non va dichiarato verde finché non termina. I tre test dichiarati preesistenti non sono sufficienti da soli: servono anche test di comportamento per il claim viewed-only e per la conservazione delle preferenze.

Patch consigliate

Patch A, priorità P0

Rimuovere `activateTimersForUser` dalle due route di polling. Lasciare il solo heartbeat. Aggiungere la chiamata a `response-timer/viewed` nel componente `ResponseNeededSlot`, condizionata a `isCurrentUser`, dopo il mount della card.

Patch B, priorità P1

Sostituire ogni `INSERT OR REPLACE` dei cooldown con conflict update parziale. Aggiungere una migrazione SQL versionata per `preference_type` ed `expires_at`, con controllo dello schema prima e dopo.

Patch C, priorità P1

Aggiungere il controllo admin fail-closed nelle tre server actions indicate, e testare che un non-admin non provochi alcuna query di scrittura.

Criteri di accettazione P0

1. `node scripts/assert-no-legacy-timer-activation.mjs` passa.
2. Polling della lega A non modifica timer della lega B.
3. La card di risposta montata per il proprietario produce un solo claim atomico.
4. Due tab concorrenti producono una sola deadline e una sola notifica.
5. Un altro utente o un utente non autorizzato non può attivare il timer.
6. Una chiusura pagina prima del mount non crea alcuna deadline.
7. Test, type-check e build passano su una revisione pubblicata, non solo sul branch locale.

Nota importante

Il report fornito dall'utente descrive fix corretti, ma il repository pubblico consultato non li mostra ancora su `main`. Prima di chiudere l'audit, pubblicare il branch/commit effettivo oppure fornire il suo SHA; altrimenti si rischia di certificare la documentazione invece del codice.